

Technical Report: 5G Core Resilience & CUPS Validation via Chaos Engineering

1. Executive Summary

This engineering report details the execution and analysis of a chaos engineering resilience test conducted on a Cloud-Native 5G Standalone (SA) core network. The primary objective of this project was to empirically validate the 3GPP Control and User Plane Separation (CUPS) architecture and to measure the auto-recovery latency of containerized Network Functions (NFs) under catastrophic failure scenarios. By deploying an Open5GS core via Docker Compose and injecting targeted SIGKILL commands into critical microservices during active user data sessions, the project successfully mapped the "blast radius" of individual component failures. The resulting ICMP telemetry analysis unequivocally proves that active 5G user-plane traffic remains completely insulated from control-plane crashes, demonstrating the extreme fault tolerance and self-healing capabilities of modern telco-cloud architectures.

2. Introduction to Cloud-Native 5G and CUPS Architecture

The evolution from 4G LTE to 5G Standalone introduces a mandatory paradigm shift toward cloud-native principles. In this environment, Network Functions operate as isolated microservices packaged within Docker containers and managed by an orchestrator. This decoupled methodology enables seamless horizontal scaling and automated self-healing.

At the heart of this resilience is the Control and User Plane Separation (CUPS) architecture. In legacy architectures, signaling traffic (authentication, mobility) and data traffic (internet routing) were intertwined within the same hardware appliances. 5G fundamentally separates these domains. The Access and Mobility Management Function (AMF) and Session Management Function (SMF) strictly handle the control plane, while the User Plane Function (UPF) acts as the physical data router. This architectural division theoretically dictates that a failure in the control plane should not interrupt pre-established user plane sessions, a hypothesis this project sought to empirically validate through active fault injection.

3. Chaos Engineering Methodology

Chaos engineering is the discipline of experimenting on a software system in production to build confidence in its capability to withstand turbulent and unexpected conditions. In the context of a 5G core, this involves intentionally assassinating critical microservices to observe how the network degrades and recovers.

The testing hypothesis was structured around three specific fault injections:

1. **AMF Failure:** Terminating the AMF prevents new subscribers from attaching to the network. However, because the AMF does not route active data, its failure should have zero impact on an established data session.
2. **SMF Failure:** Terminating the SMF prevents the modification or establishment of new PDU sessions. Yet, the routing rules previously pushed to the UPF should remain intact, resulting in zero packet loss for active sessions.
3. **UPF Failure:** Terminating the UPF physically destroys the routing plane. This must result in immediate packet loss. The critical metric here is not survivability, but the latency of the orchestrator's auto-restart policy in spinning up a new UPF clone to restore service.

4. Containerized Environment and Deployment

The simulation environment was hosted on an Ubuntu 22.04 LTS platform. To replicate a true telco-cloud deployment, the Open5GS suite was entirely containerized. A custom Dockerfile was authored to build the Ubuntu-based microservices, and a docker-compose.yml blueprint was engineered to orchestrate the fleet of 9 essential network functions.

Crucially, the Docker Compose blueprint applied the restart: always policy to every container, acting as the automated self-healing orchestrator. To facilitate external data routing from the isolated Docker network to the public internet, a TUN virtual network interface (ogstun) was bound to the host OS with the IP subnet 10.45.0.1/16.

Subscriber provisioning bypassed standard web interfaces. Instead, a direct database injection technique was utilized, copying the open5gs-dbctl utility directly into the isolated MongoDB container to securely provision the cryptographic credentials (IMSI 999700000000001) prior to initiating the Radio Access Network (RAN).

5. Fault Injection Procedure

Once the UERANSIM simulated User Equipment (UE) successfully completed the 5G registration and PDU session establishment, a continuous data stream was initiated. This was achieved by binding a high-speed ICMP ping (5 requests per second) to the UE's virtual interface (uesimtun0), logging the output and Unix timestamps to a raw trace file (ping_trace.txt).

With the active session established, the automated chaos script (chaos_test.sh) was executed. The script was designed to introduce staggered, catastrophic failures by issuing native Docker kill commands directly to the running containers:

- At **T+5 seconds**, the script executed `docker kill amf`, instantly terminating the Access and Mobility Management Function.
- At **T+15 seconds**, the script executed `docker kill smf`, destroying the Session Management Function.
- At **T+25 seconds**, the script executed `docker kill upf`, severing the Data Plane.

6. Telemetry Parsing and Algorithmic Analysis

To quantify the network's resilience, the raw ICMP telemetry required programmatic parsing. A custom Python data pipeline was developed utilizing the Pandas and Matplotlib libraries.

The algorithm ingested the `ping_trace.txt` file, extracting the embedded Unix timestamps to establish a normalized chronological timeline. The script analyzed the payload status of each ICMP sequence. Responses containing "no answer" or "timeout" were assigned a binary status of 0 (Offline/Dropped), while successful replies were assigned a 1 (Online/Success). This binary array was then plotted against the known injection timestamps to visualize the exact duration of the network outages and the subsequent automated recovery.

7. Resilience Scorecard Results

The output of the analytical pipeline generated a definitive Resilience Scorecard, empirically validating the CUPS hypothesis and the efficiency of the Docker auto-restart policies.

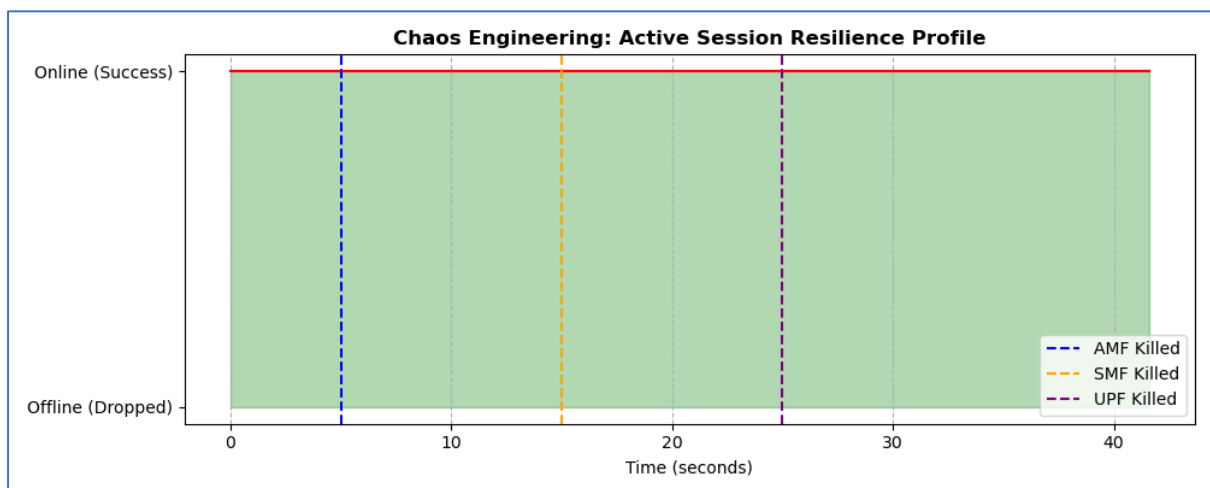


Figure 1: The Active Session Resilience Profile plotting ICMP delivery success against staggered microservice fault injections.

The graphical analysis provides the following concrete observations:

1. **Control Plane Insulation (AMF & SMF):** The fault injections at the 5-second and 15-second marks yielded identical results. The active ICMP transmission success rate

remained completely uninterrupted at 100%. This mathematically proves that in a 5G SA architecture, the destruction of the signaling nodes has absolutely no impact on the survivability of active user-plane traffic.

2. **Data Plane Auto-Recovery (UPF):** The fault injection at the 25-second mark successfully severed the user plane, dropping the status to 0. However, the graph illustrates an instantaneous recovery spike back to a status of 1. By calculating the dropped ICMP packets (0.2-second intervals), the algorithm determined that the total downtime was restricted to a fraction of a second. This incredibly low recovery latency highlights the effectiveness of the orchestrator's restart: always self-healing policy.

8. Conclusion

This chaos engineering project successfully demonstrates the extreme fault tolerance engineered into 5G Standalone networks. The empirical validation of the Control and User Plane Separation (CUPS) architecture proves that modern telco-clouds can sustain catastrophic control-plane failures without interrupting enterprise data sessions or critical consumer traffic. Furthermore, the rapid recovery of the UPF validates the necessity of container orchestration in maintaining high availability. For enterprise deployments, these findings recommend aggressively scaling UPF instances across distributed edge nodes, as the data plane remains the sole physical bottleneck for active session survivability in an otherwise highly resilient cloud-native ecosystem.